

Truewise

A public-data platform that turns scattered federal education data into clear, honest guidance for students and families

Detailed project plan · Prepared June 18, 2026 · Name: Truewise · Domain: truewise.us

The idea in one line. Build one shared data warehouse over the major federal education datasets, joined on common school identifiers, then ship a family of guidance tools (modules) on top of it: college affordability, ROI, social mobility, transfer planning, K-12 opportunity, and career pathways. The hard part is built once; every tool reuses it.

1. Vision and mission

Every module shares one mission: replace the scary or hidden number with the real one. Students rule out affordable colleges because they only see sticker price, miss aid they qualify for, pick majors on anecdote, and overlook schools that quietly change lives. Truewise exists to give them honest numbers from public data, presented simply. The framing is positive and opportunity-opening rather than problem-cataloguing: the goal is for a user to find or claim something they would otherwise have missed.

2. The unifying architecture

These tools generalize cleanly because the major datasets share join keys. Build one warehouse, key everything correctly, and each "project" becomes a different question asked of the same tables.

Entity	Join key	Links together
Colleges	IPEDS UnitID (and OPEID)	College Scorecard, IPEDS, Federal Student Aid, Opportunity Insights
K-12 schools / districts	NCES school and district IDs	Common Core of Data, Civil Rights Data Collection, SEDA
Geography	FIPS / ZIP / state	Census ACS, labor data, everything above

The shape is a shared data spine plus interchangeable modules. One ingestion-and-refresh pipeline pulls the bulk files, cleans them, and lands them in DuckDB / Parquet. Every module reads from that one source. Add a dataset once and any number of modules can use it.

3. Data sources

Dataset	Provides	Key	Refresh
College Scorecard	Net price by income, tuition, debt, completion, field-of-study earnings, repayment, Pell outcomes	UnitID	Annual
IPEDS	Enrollment, admissions, transfer, finance, program offerings	UnitID	Annual
Federal Student Aid	FAFSA completion, loan volumes, default/repayment	OPEID	Annual/ quarterly
Opportunity Insights	College social-mobility rates (low-income to higher-income)	UnitID/super-OPEID	Static releases
NCES Common Core of Data	K-12 school/district directory, enrollment, staffing	NCES ID	Annual

Civil Rights Data Collection	Advanced-course access, discipline, dual enrollment	NCES ID	Biennial
SEDA (Stanford)	K-12 achievement and growth, comparable across districts	NCES ID	Periodic
BLS + O*NET	Occupation pay, growth projections, skills	SOC code	Annual
Census ACS	Income, demographics, geography	FIPS	Annual

All are free and publicly downloadable in bulk. College Scorecard data was last refreshed June 10, 2026.

4. Data footprint, storage, and compute

This is "medium data," not "big data." Almost everything is plain tabular CSV: one row per school, or per school-and-program, or per school-and-year. The widest file (College Scorecard institutions) is short but wide (~6,500 rows, ~3,000 columns); the largest by row count are the K-12 files at roughly 100,000 schools. The biggest single table you will touch is a few million rows, which DuckDB aggregates in well under a second.

Dataset	Approx raw size	Note
College Scorecard (Most Recent files)	~150–250 MB	What you actually need
College Scorecard (all years, raw)	~1–2 GB	Only if you want history
IPEDS (selected surveys)	~few hundred MB	Pull subsets, not everything
NCES Common Core of Data	~few hundred MB	K-12 directory + enrollment
Civil Rights Data Collection	~34 MB zipped / yr	Surprisingly small
SEDA	~hundreds MB	Test-score archive
Opportunity Insights	tens of MB	Small
BLS / O*NET	tens of MB	Small
Census ACS (selected tables)	~hundreds MB—~2 GB	The one that can balloon; pull only needed variables

Disk: grabbing everything is ~3–8 GB of raw downloads. Converted to Parquet (columnar + compression), the working footprint drops to ~1–3 GB. Keep ~30–40 GB free during the build as scratch room for downloads and unzips; steady-state is just a few GB.

Compute and RAM: the workload is read-heavy analytics (load, join on school IDs, aggregate, rank). DuckDB runs entirely on the laptop, no server, and streams/spills to disk when data exceeds RAM, so it handles tens of GB on an 8 GB machine. The optional scikit-learn model trains on thousands to low-millions of rows in seconds to a couple minutes on CPU. No GPU is ever needed.

Hardware verdict: a MacBook Air handles this comfortably. An 8 GB Air is fine (queries finish in under a second to a few seconds); 16 GB just adds headroom. The only real constraints are disk space during the initial download and pulling Census ACS selectively. The bottleneck on this project will be design time, not hardware.

5. Module catalog

Group modules by who they serve so the site reads as coherent navigation, not a pile of features. All run off the shared spine.

Section	Modules	Primary data
Choosing & paying for college	Affordability matcher (net price by income), ROI analyzer, Value Check (earnings-premium & debt-to-earnings warning), social-mobility index, loan-repayment-risk view	College Scorecard, IPEDS, FSA, Opportunity Insights
Career & pathways	Major-to-career outcomes explorer (CIP→SOC crosswalk), graduate-program ROI, apprenticeship finder (optional; separate DOL data, not the shared spine)	College Scorecard field-of-study, BLS, O*NET, DOL
K-12 opportunity	Advanced-course access, "beating the odds" schools, dual enrollment access, FAFSA completion gaps	CRDC, NCES CCD, SEDA, FSA

Connective tissue: a shared **school/program profile page** that any module can deep-link into. The affordability tool and the ROI tool both point at the same college's page. This is what makes Truewise one product rather than a dashboard junk drawer.

Timely opportunity (build early): the federal Financial Value Transparency & Gainful Employment framework takes effect July 1, 2026, judging every program on two tests — a debt-to-earnings ratio and an earnings premium (do graduates out-earn a typical high-school graduate in their state). College Scorecard *already* publishes the earnings-premium comparison (state and national), and the 2026–27 FAFSA will flag schools whose graduates earn less than a typical high-school grad. The **Value Check** module turns this into a plain-language warning, mirrors the government's own accountability metrics, and needs no new dataset. It is the sharpest "honest number" feature on the platform.

Deliberately out of scope: a true *transfer-credit planner* (which courses transfer where) is not buildable from federal data alone — those are state articulation agreements, absent from IPEDS, which only reports transfer-out counts. Cut to avoid overpromising. Apprenticeship search uses a separate DOL system (Apprenticeship.gov / RAPIDS), so it stays an optional bolt-on rather than part of the shared spine.

6. Tech stack

Layer	Choice	Why
Ingestion	Python (requests, pandas)	Pull bulk files, normalize, validate
Storage / query	DuckDB + Parquet	Fast columnar analytics, no server; handles the large K-12 and prescriber-scale files in memory; DuckDB-WASM can run in the browser
Transform	SQL in DuckDB (+ optional dbt)	Keep join-key logic and module queries versioned and testable
Analysis / modeling	pandas, scikit-learn	Derived metrics; optional earnings estimation for thin cohorts
Geospatial	GeoPandas	Maps of access and outcomes
Backend (optional)	FastAPI	Only if a module needs live server-side queries; a real full-stack growth step
Frontend	React (or lightweight vanilla JS) + a charting lib	Builds on your prior HTML / Web Speech API work; shared component library across modules
Charts / maps	Recharts or Chart.js; MapLibre for maps	Free, capable, no API keys for base layers
Automation	GitHub Actions	Annual full refresh + monthly update checks
Testing	pytest + data-validation checks	Catch schema drift and broken joins on every refresh

7. Architecture: two paths (and the recommended hybrid)

Dimension	Path A — Static precompute	Path B — FastAPI backend
How	CI computes all module outputs, writes Parquet/JSON (or ships DuckDB-WASM); browser queries static files	FastAPI serves live queries from DuckDB; React calls the API
Backend hosting	None	Render free tier or ~\$5/mo
Cold starts	None	Yes on free tier (30–60s)
Fit	Excellent: data is static between annual refreshes	Useful only for heavy/interactive cross-filtering
Growth	Strong data-engineering + frontend	Adds full-stack API + deployment
Cost	\$0	\$0–\$5/mo

Recommended: hybrid, static-first. Precompute every module in CI and serve a static site (Path A) for almost everything, since the data only changes once a year. If one module later needs heavy live cross-filtering, add a single FastAPI endpoint (Path B) just for that, without moving the rest. You get \$0 hosting and no cold starts now, and a clean upgrade path to full-stack experience when you want it.

8. Hosting and infrastructure

Component	Choice	Cost
Code + data pipeline	GitHub repo + GitHub Actions	\$0
Static site (frontend + precomputed data)	Cloudflare Pages or Netlify or GitHub Pages	\$0
Optional backend	Render free tier (cold starts) or ~\$5/mo always-on	\$0–\$5/mo
Large data artifacts	Release assets / Cloudflare R2 free tier if needed	\$0
Custom domain (optional)	e.g. edcompass.org	~\$12/yr

2026 reality: backend free tiers tightened. Fly.io dropped its free tier for new accounts; Railway is now effectively a one-time \$5 trial then \$1/mo for a tiny service; Render's free web service spins down after 15 min idle and cut bandwidth to 5 GB/mo. Static hosting sidesteps all of it, which is why the hybrid leans static.

9. Cost summary

Scenario	Monthly	Yearly
Static-only (recommended start)	\$0	\$0 (≈\$12 with a domain)
Static + one free-tier backend module	\$0	≈\$12 with a domain
Static + always-on backend	~\$5	~\$60–\$72 with a domain

Data, compute, CI, and frontend hosting are all free. The only money is an optional always-on backend and an optional domain.

10. Build phases and steps

Build the spine first with just enough data for one module, ship that module end to end, then add modules incrementally. Each new module is mostly a new query plus a new view, not new infrastructure. Effort is in focused part-time days.

Phase	Steps	Effort
0. Foundation	Repo, structure, environment, CI skeleton, decide naming/branding	1 day
1. Data spine v1	Ingest College Scorecard bulk files; build the college identity layer (UnitID/OPEID); land in DuckDB/Parquet; add schema + validation tests; document fields and suppressed-value handling	3–4 days
2. First module: Affordability matcher	Net-price-by-income queries; ranking logic; build the shared school profile page; build the income/filter input UI; charts	4–6 days
3. Design system	Shared UI components, navigation shell, methodology/caveats page, "your situation in → honest numbers out" pattern reused by all modules	2–3 days
4. ROI + Value Check module	Reuse spine; field-of-study earnings/debt; payback + completion-weighted ROI; Value Check flags programs whose grads earn below a typical high-school grad (state + national) and surfaces debt-to-earnings, using Scorecard's existing earnings-premium fields; plug into profile page	4–5 days
5. Social mobility module	Add Opportunity Insights dataset to spine; mobility index; "hidden gems" view	2–3 days
6. More college modules	Loan-repayment-risk view (adds FSA data); fold Pell-success into the affordability + mobility views rather than a separate module	3–5 days
7. Career section	Add BLS + O*NET; major-to-career explorer (CIP→SOC); grad ROI; apprenticeship finder optional (separate DOL data)	4–7 days
8. K-12 section	Add NCES CCD + CRDC + SEDA (second join key); advanced-course access, beating-the-odds, dual enrollment, FAFSA completion	7–10 days
9. Refresh automation	GitHub Actions: annual full refresh + monthly update checks; alerting on schema drift	1–2 days
10. Polish + launch	Accessibility, performance, About/methodology, SEO, analytics, public launch	2–3 days

Effort by scope

Scope	Includes	Total (part-time)
MVP	Phases 0–3 (spine + affordability + design system)	~2–3 weeks
Strong college product	Add ROI, social mobility, 1–2 more college modules, refresh automation	~5–7 weeks
Full platform	All three sections, all modules, polish + launch	~10–14 weeks

11. Risks and mitigations

Risk	Mitigation
	Show "insufficient data" explicitly; never silently impute

Suppressed / missing data (small cohorts)	
Data lags reality by 1–2 years	Label cohort years; frame numbers as "students like you paid/earned roughly"
Join-key mismatches across datasets	Build a documented crosswalk in phase 1; validation tests on every refresh
Scope creep across 12+ modules	Ship MVP first; treat each module as an independent, optional increment
Over-claiming (tool as verdict)	Consistent caveats; point users to official Net Price Calculators / advisors for personal numbers
Large K-12 files	DuckDB columnar queries; precompute outputs so the browser never loads raw files

12. Roadmap at a glance

Milestone	What's live
M1 — MVP	Data spine + affordability matcher + shared profile page
M2 — College suite	ROI, Value Check, social mobility, repayment risk
M3 — Careers	Major-to-career, grad ROI, apprenticeships
M4 — K-12 + launch	Course access, beating-the-odds, dual enrollment, FAFSA gaps; public launch

13. Suggested first steps

- Create the repo and CI skeleton; lock a working name and a one-line mission.
- Build the College Scorecard ingestion and the college identity layer (phase 1); confirm you can join institution and field-of-study tables on UnitID.
- Ship the affordability matcher end to end for one income bracket and a handful of schools, including the shared profile page, before scaling.
- Stand up the design system early so every later module slots into a consistent shell.
- Decide static-only vs hybrid only after the first module, once you know how interactive it needs to feel.

Sources: College Scorecard data and documentation (collegescorecard.ed.gov, last updated June 10, 2026); IPEDS, NCES Common Core of Data, Civil Rights Data Collection, Federal Student Aid, Opportunity Insights, BLS/O*NET, Census ACS (all public). 2026 hosting free-tier status for Render, Railway, and Fly.io.